ENGINEERING TIMELINE · DEFERRED TRANSFER

Deferred Transfer Rollout

A milestone-by-milestone timeline and task plan for shipping deferred operator transfers on 2027+ bookings.

Scope: deferred transfer engine only — not the multi-operator NAT Network extension.

ENGINEERING TIMELINE · INTERNAL

Prepared for
NAT Leadership

Prepared by
Product & Engineering

Date
September 2026

Basis
2027 Deferred Transfer plan doc

THE PLAN AT A GLANCE

A 12-week rollout, six milestones

Every task below is sized from the actual engineering surface area — the checkout flows, refund paths, and services this change touches — not a rough guess.

38	≈65	12	2–3
TASKS	PERSON-DAYS EFFORT	WEEKS, ELAPSED	ENGINEERS ASSUMED

ROADMAP (SEQUENTIAL MILESTONES)

1	2	3	4	5	6

1 **Foundation**
Week 1

2 **Transfer Engine**
Weeks 2–3

3 **Platform-Wide Rollout**
Weeks 4–6

4 **Refunds & Cancellations**
Weeks 7–9

5 **Reporting**
Week 10

6 **Kill Switch & Launch**
Weeks 11–12

Milestones are sequential (each depends on the last); tasks within a milestone are parallelizable across the team — see Milestone 3 and 4 in particular, both sized for 2–3 people working different flows at once.

What this covers

Moving how card and card-reader payments settle for any tour departing 2027 or later — from an automatic Stripe destination-charge split to an explicit, code-driven Transfer to the operator’s connected account. Same guest experience, same operator payout timing, same pricing; only the underlying settlement mechanism and its visibility change.

M1 Foundation

Recognize a 2027+ booking at the moment it’s paid for, and lock in which payment mode it uses, permanently.

Week 1 · 4d effort

TASK	NOTE	EST.
payment_mode column + migration	Immutable per booking; never recomputed from the current tour date.	**1d**
Build PaymentModeResolver	Date gate at 2027-01-01, tenant-local; single-date and plural (combo) forms.	**1.5d**
Combo-cart rule	Any row ≥ 2027 forces the whole cart to deferred mode.	**1d**
Resolver unit tests	Boundary date, mixed input types, combo arrays, null handling.	**0.5d**

M2 Transfer Engine

Build the engine that sends an operator their share as its own explicit Transfer call, modeled on the existing partner-split engine.

Weeks 2–3 · 7.5d effort

TASK	NOTE	EST.
transfers ledger table + transfer_type	New primary_operator discriminator alongside the existing partner_split rows.	1d
transferToOperator() + reverseOperatorTransfer()	Non-partner path: send + reverse the operator’s share.	2d
transferSellingCommission() + reversal	Partner-package selling-tenant path — Partner Fee only, not the ticket-cost remainder.	1.5d
Harden existing lookups with transfer_type filter	Stops partner-split and primary-operator rows cross-contaminating each other’s queries.	1d
Currency fix	Transfer amount uses the tenant’s real currency, not a hardcoded USD.	0.5d
Engine unit tests	Card success, cash no-op, Stripe failure handling, over-reversal guard, zero-fee case.	1.5d

M3 Platform-Wide Rollout

Wire every checkout flow — web, mobile, admin, card reader, POS — to resolve mode and fire the transfer correctly.

Weeks 4–6 · 22.5d effort

TASK	NOTE	EST.
Branch the shared PaymentIntent param builder on mode	buildConnectedPaymentData() — the one place every creation site already funnels through.	1d
New Booking + Make Payment (web)	The two highest-traffic creation/confirmation flows.	2d
Combo cart checkout	Multi-package cart sharing one PaymentIntent.	1.5d
Change People / Change Payment / Switch Package	Shared edit-booking helper plus Switch Package's own flow.	2.5d
Card Reader (Terminal)	Shared creation endpoint + tour-date plumbing for New Booking/POS reader tabs.	2d
Point of Sale (POS) checkout	In-person combo-cart-capable creation + confirmation.	1.5d
Tip, Edit Discount, Custom Charge, Add-Ons, Addon Edit Charge	Five features sharing one creation helper — wire once, confirm each.	3d
Payment Links checkout	Checkout-session based, mode carried in session metadata.	1.5d
Mobile API v2 (new booking + 3D Secure)	App checkout and its 3DS completion callback.	2d
Legacy (API v1) creation-method audit	Sweep every other legacy PaymentIntent-creation method for the same gap.	2.5d
End-to-end tests, one per flow	Real booking, real call, asserting on real DB state — not just a code read.	3d

M4 Refunds & Cancellations

Make every refund, cancellation, and dispute flow correctly reverse a deferred-mode transfer, matching today's behavior.

Weeks 7–9 · 19d effort

TASK	NOTE	EST.
buildRefundParams() mode-aware helper	Strips reverse_transfer / refund_application_fee for deferred mode.	1d
DisputeService dispute-loss branch	Reverses the separate primary_operator transfer when a dispute is lost.	1d
"Refund Order" flow (Ticket + Weather refund)	PosRepository's refund dispatch, both no-partner-handling services.	2d
Cancel Booking flow (3 services)	Ticket / Weather / Partial refund-on-cancel — including the partial-refund spillover split.	3d
Add-ons refund sites	All 4 refund call sites in the shared add-ons repository.	2d
Edit Booking + Custom Charge refund sites		1.5d
Legacy + V3 TourBookingRepository sweep	The largest single file — refund(), editDiscountRefund(), refundCustomCharge(), editBooking(), deletePeopleChangePeople().	4d
Regression pass across existing refund/add-on suites	Catch any mocking-hazard regressions from the new interface method.	1.5d
End-to-end tests, one per refund flow		3d

M5 Reporting

Give the team visibility into any transfer that failed, so it gets manual follow-up instead of going unnoticed.

Week 10 · 3.5d effort

TASK	NOTE	EST.
Failed-transfer admin view	Surfaces any Failed ledger row for manual follow-up.	**2d**
Alerting for a failed transfer	Email or dashboard flag — no silent failures.	**1d**
Tests		**0.5d**

M6 Kill Switch & Launch

Ship a per-tenant on/off switch, rehearse end-to-end in Stripe test mode, and roll out gradually with a rollback plan.

Weeks 11–12 · 8d effort

TASK	NOTE	EST.
Per-tenant DEFERRED_TRANSFER_ENABLED switch	Same mechanism as the existing AUTO_PAYOUT toggle, plus its admin UI.	**1.5d**
Full regression suite run + fixes	Every milestone's tests, together, one final time.	**1.5d**
Stripe test-mode end-to-end rehearsal	Simulated real bookings, refunds, and disputes before go-live.	**2d**
Rollout runbook + rollback plan		**1d**
Staged go-live	Enable for one pilot operator, monitor, then expand.	**2d**

READING THIS TIMELINE

Assumptions & notes

What this schedule does and doesn’t assume, stated plainly.

Assumptions behind the 12-week estimate

- Team size: roughly 2–3 engineers, with tasks inside a milestone split across people where the work is genuinely parallel (e.g. Milestone 3’s eleven checkout flows, Milestone 4’s nine refund paths).
- Task estimates are engineering effort in person-days, not elapsed calendar time — milestone durations account for parallelization, review, and integration overhead on top of the raw effort sum.
- Milestones are sequential: each depends on the work before it (the transfer engine needs the payment-mode gate; every checkout flow needs the transfer engine; refunds need the checkout flows wired first).
- No 2027-dated tour has been sold yet, so this entire timeline can run in the background with zero risk to a real guest or booking already in progress.

What could move the date

- Fewer than 2–3 engineers available, or engineers splitting time with other work, extends Milestones 3 and 4 the most — they carry the bulk of the effort.
- Milestone 5 (Reporting) can start in parallel with the tail end of Milestone 4 if resourcing allows, tightening the schedule by up to a week.
- A real Stripe-account or connected-account issue surfaced during Milestone 6’s rehearsal would extend that milestone specifically, not the ones before it.